

Course Project Report

Multiscale and Multilayer Contrastive Learning for Domain Generalization

Project Implementation, Evaluation, and Experimental Discussion

Group: 26
Members: Vu Thuong Tin
Chu Anh Duc
Nguyen Xuan Khai
Topic: Domain Generalization in Computer Vision
Method: M²-CL
Backbones: ResNet-18, ResNet-50 (ImageNet pretrained)
Benchmarks: PACS · VLCS · Office-Home

June 18, 2026

Contents

Abstract	1
1 Introduction	1
1.1 Project Motivation	1
1.2 Report Scope	2
1.3 Main Contributions of This Report	2
2 Problem Formulation	2
3 Related Work	2
4 Proposed Method	3
4.1 Backbone Architecture	3
4.2 Extraction Block and Concentration Pipelines	4
4.3 Contrastive Regularization	5
5 Implementation in the Project Codebase	6
5.1 Repository Organization	6
5.2 Model Construction	6
5.3 Extraction Block Details	6
5.4 Loss Implementation	7
5.5 Training Pipeline	7
5.6 Configuration Defaults	7
6 Project Workflow	7
7 Experimental Setup	7
7.1 Datasets	7
7.2 Training Configuration	8
7.3 Baselines	8
8 Quantitative Results	8
8.1 Summary Results	8
8.2 Breakdown by Comparison	9
8.3 Per-Dataset Interpretation	9
9 Ablation and Sensitivity Analysis	9
9.1 Key Lessons from the Ablation	10
10 Qualitative Results	10

11 Discussion	11
12 Limitations	12
13 Conclusion	12
A Full Result Tables	12
B References	14

Abstract

Domain generalization (DG) studies the problem of training a visual recognition model on a set of source domains and deploying it on unseen target domains without access to target-domain data during training. This report presents our course project on M²-CL, a multiscale and multilayer contrastive learning framework for visual domain generalization. The method extends an ImageNet-pretrained ResNet backbone with extraction blocks attached to intermediate convolutional layers. Each extraction block processes feature maps through multiple parallel concentration pipelines at several spatial scales, compressing and pooling them into a compact representation. A supervised contrastive regularizer is then applied to intermediate representations so that same-class samples are pulled together while different-class samples are pushed apart. We evaluate the method on PACS, VLCS, and Office-Home under leave-one-domain-out evaluation, compare it against ten DG baselines, and analyze both quantitative accuracy and qualitative saliency-map behavior. The experimental results show that M²-CL improves over both the strongest baseline and the M² architecture on the selected benchmarks under ResNet-18 and ResNet-50.

1. Introduction

Modern image classifiers achieve strong in-distribution performance, but their accuracy drops sharply when test images come from a visual context absent during training. This is common in practice: a model trained on photographs may fail on sketches, artworks, cartoons, or images captured under different lighting or camera conditions. The root cause is that empirical risk minimization (ERM) does not distinguish between *class-relevant evidence* and *domain-specific shortcuts*. If a training domain repeatedly associates a class with a background, texture, or color distribution, a high-capacity CNN can exploit that correlation while still achieving low source-domain error.

Domain generalization addresses this failure mode under a strict constraint: target-domain images and labels are unavailable during training. A DG model must therefore learn representations that remain useful under distribution shift by relying only on the diversity present in source domains.

Our project studies a multiscale and multilayer contrastive learning framework for domain generalization [1]. The central idea is that intermediate CNN features contain rich class-discriminative information that can be lost or entangled with domain-specific attributes in the final feature vector. The model therefore extracts representations from multiple layers and multiple spatial scales, then regularizes them with a supervised contrastive objective that promotes class-level invariance across source domains.

1.1 Project Motivation

This project focuses on building a complete working system rather than only describing the model at a high level. Besides mathematical formulation and final benchmark numbers, the report answers practical implementation questions:

- What exact problem is being solved, and why is the standard supervised setting insufficient?
- How is the method implemented as concrete PyTorch modules and training scripts?
- How are datasets, domains, training splits, checkpoints, and evaluation metrics organized?
- Do the experimental results support the intended conclusions, and where are the uncertainties?

- What practical issues appear when training the model end to end?

1.2 Report Scope

The current repository and this report focus on three domain-generalization benchmarks: PACS, VLCS, and Office-Home. The comparison tables include ten representative baselines: ERM, RSC, CORAL, Mixup, MMD, SagNet, SelfReg, ARM, EQRM, and SAGM.

1.3 Main Contributions of This Report

1. Explains the DG setting with explicit source-domain and target-domain notation.
2. Maps the architecture to concrete PyTorch modules, forward hooks, extraction blocks, and training scripts.
3. Presents the contrastive loss used by the project and explains how it is implemented.
4. Reports and interprets the project’s experimental results on the selected datasets.
5. Adds practical observations about memory usage, batch composition, and training stability.

2. Problem Formulation

Let $\mathcal{D} = \{D_1, D_2, \dots, D_N\}$ be a set of source domains. Each domain contains image-label pairs (x, y) drawn from a domain-specific joint distribution over $\mathcal{X} \times \mathcal{Y}$. Under the leave-one-domain-out protocol, one domain is held out as the unseen target domain while the remaining domains form the training and validation set. The goal is to learn a classifier $f_\theta : \mathcal{X} \rightarrow \mathcal{Y}$ that minimizes target-domain error without ever observing target-domain data during training.

The central challenge is *representation entanglement*. A learned feature vector may capture both class-specific attributes (object shape, discriminative parts) and domain-specific attributes (sketch strokes, cartoon colors, background scenes, lighting, or image texture). If the classifier relies on the latter, it will fail when the target domain shifts. The desired behavior is to produce features that preserve class information while suppressing unstable domain-specific correlations.

3. Related Work

Several families of methods have been proposed for DG, as summarized in Table 1. ERM is the simplest baseline and trains by minimizing cross-entropy over pooled source-domain data. Regularization methods such as RSC [3] suppress dominant features to prevent shortcut reliance. Distribution alignment methods such as CORAL [4] and MMD [5] encourage source-domain feature distributions to become closer. Data-mixing methods such as Mixup [6] construct interpolated training samples. Style-oriented methods such as SagNet [7] reduce dependence on domain-specific style.

M²-CL differs from these approaches by explicitly reusing intermediate convolutional features and regularizing them with a supervised contrastive objective. This design is motivated by the insight that early and middle CNN layers still contain spatial and local visual evidence that may be lost or entangled by the final global representation.

Table 1: High-level comparison of DG method families used in the project.

Family	Representative methods	Main idea
Standard supervision	ERM	Pool all source domains, minimize cross-entropy.
Feature suppression	RSC	Hide highly predictive features to reduce shortcut reliance.
Distribution alignment	CORAL, MMD	Penalize differences between source-domain feature distributions.
Data interpolation	Mixup	Train on convex combinations of samples and labels.
Style robustness	SagNet	Reduce dependence on style-related information.
Self/extra regularization	SelfReg, SAGM	Auxiliary constraints for smoother or more stable features.
Risk-based methods	ARM, EQRM	Modify the risk objective to improve robustness.
Multilayer extraction	M ² , M ² -CL	Use intermediate CNN features with contrastive class alignment.

4. Proposed Method

4.1 Backbone Architecture

The method uses ImageNet-pretrained ResNet-18 and ResNet-50 backbones. A standard ResNet classifier passes the final convolutional output through global average pooling and a linear classifier. In contrast, M² attaches *extraction blocks* to selected intermediate layers. These blocks collect feature maps at several depths, compress them, process them at multiple spatial scales, and concatenate the resulting vectors into a hypercolumn-like representation that is then classified by a single fully-connected head.

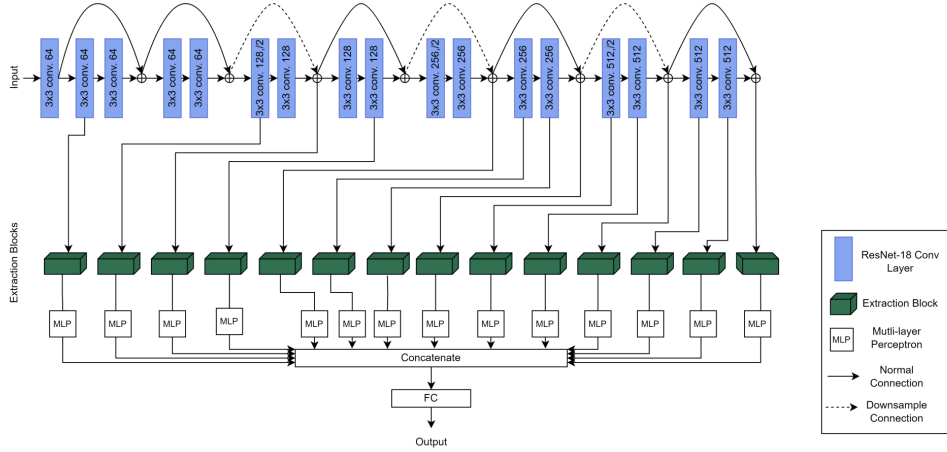


Figure 1: High-level architecture of M². Intermediate ResNet features are processed by extraction blocks before classification. The solid lines denote same-dimensional connections; dashed lines denote downsampled connections.

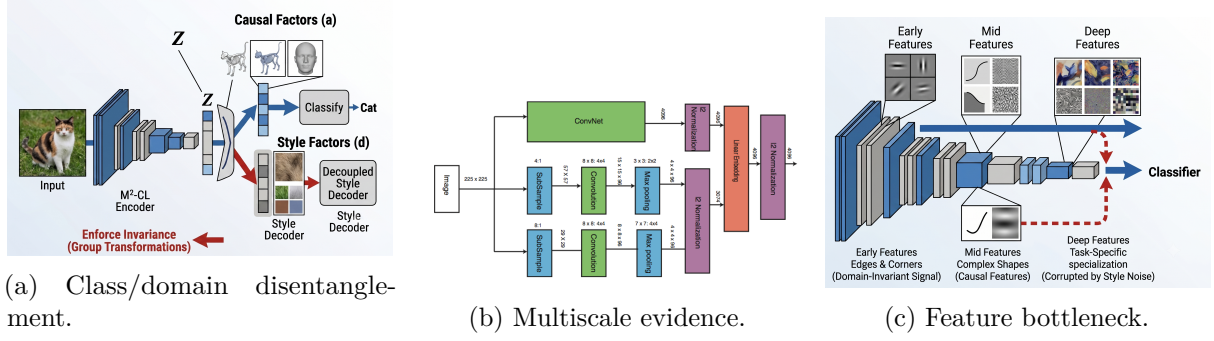


Figure 2: Visual intuition behind the method: disentangle class and domain attributes, collect information at several spatial scales, and compress intermediate maps before classification.

4.2 Extraction Block and Concentration Pipelines

An extraction block receives an intermediate feature map $F^{(l)} \in \mathbb{R}^{C_l \times H_l \times W_l}$ from layer l . It consists of a set of parallel *concentration pipelines*, each of which applies the following sequence of operations:

1. A 1×1 convolution that reduces the channel count by a reduction factor r , producing $\lfloor C_l/r \rfloor$ channels.
2. A Dropout2d (spatial dropout) layer that drops entire feature maps rather than individual pixels.
3. A $k \times k$ max-pooling layer with stride 1. The pool size k is pipeline-specific: for early layers, the three pipelines use $k \in \{8, 4, 2\}$; for layers in the last residual block, two pipelines use $k \in \{7, 3\}$.
4. ReLU activation and flattening of the spatial grid.
5. A two-layer MLP that projects the flattened vector into a compact representation of fixed dimension.

Our implementation uses **3 concentration pipelines per extraction block in early layers** and **2 pipelines per block in later layers**, for both ResNet-18 and ResNet-50. The outputs of all pipelines from all extraction blocks are concatenated into a single feature vector, which is passed through a classification head.

Spatial dropout is crucial here because neighboring pixels in convolutional feature maps are highly correlated: standard dropout can remove isolated activations while leaving nearby evidence intact, which is insufficient to regularize shortcut patterns that span a spatial region. Dropping entire channels encourages the model to learn predictions that do not rely on any single feature map.

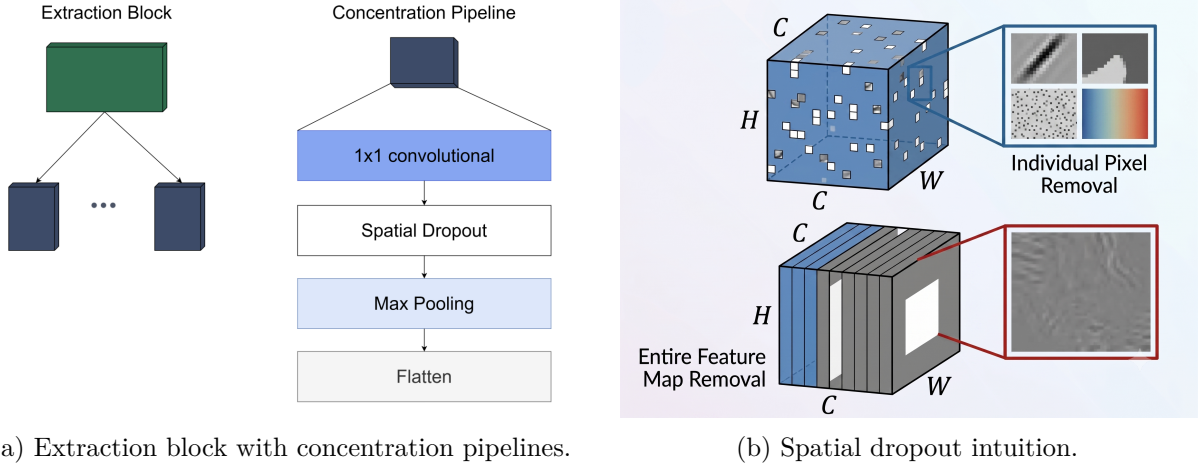


Figure 3: Core components that transform intermediate convolutional features into compact multiscale representations.

4.3 Contrastive Regularization

The full M²-CL model adds a supervised contrastive regularizer to the M² architecture. Let $\mathbf{u}_i^{(l)}$ be the L2-normalized representation of sample i extracted from layer l , with $\|\mathbf{u}_i^{(l)}\| = 1$. For a mini-batch of N_b samples, let $\mathcal{I}_c = \{i \mid y_i = c\}$ be the set of batch indices belonging to class c , and let $N_c = |\mathcal{I}_c|$.

For each layer l , the project first computes the pairwise energy

$$e_{ij}^{(l)} = \exp\left(\mathbf{u}_i^{(l)\top} \mathbf{u}_j^{(l)}\right), \quad (1)$$

and the full-batch denominator

$$Z^{(l)} = \sum_{i=1}^{N_b} \sum_{j=1}^{N_b} e_{ij}^{(l)}. \quad (2)$$

For every class that appears at least twice in the mini-batch, the class score is:

$$p_c^{(l)} = \frac{\frac{2}{\tau} \sum_{\substack{i,j \in \mathcal{I}_c \\ i < j}} e_{ij}^{(l)}}{Z^{(l)}}, \quad (3)$$

where $\tau > 0$ is the temperature hyperparameter. The numerator uses unordered same-class positive pairs and the factor 2 accounts for the two symmetric directions of each pair. The denominator sums over all ordered batch pairs, including cross-class pairs, and is shared by all classes.

The layer score is the sum of log class scores:

$$S_l = \sum_{\substack{c=1 \\ N_c \geq 2}}^C \log p_c^{(l)}. \quad (4)$$

Equivalently, the layer-wise contrastive penalty can be written as:

$$\mathcal{L}^{(l)} = -S_l = - \sum_{\substack{c=1 \\ N_c \geq 2}}^C \log p_c^{(l)}. \quad (5)$$

Minimizing this penalty pushes same-class pair similarities higher relative to the full-batch denominator. The total training objective is:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \alpha \sum_{l=1}^M \mathcal{L}^{(l)}, \quad (6)$$

where $\mathcal{L}_{\text{CE}}$ is cross-entropy, M is the number of layers from which representations are extracted, and α controls the contrastive weight. The default settings are $\alpha = 0.01$ and $\tau = 1.0$.

5. Implementation in the Project Codebase

5.1 Repository Organization

Table 2 maps the most important files to their roles.

Table 2: Mapping from project files to their roles in the implementation.

File or folder	Role in the project
<code>models/m2cl.py</code>	Builds ERM, M ² , and M ² -CL ResNet-based architectures.
<code>models/extraction_block.py</code>	Implements concentration pipelines, spatial dropout, pooling, and MLP.
<code>losses/contrastive.py</code>	Implements the layer-wise contrastive regularizer (Eq. 6).
<code>algorithms/baselines.py</code>	Wraps M ² -CL and baselines behind a common training interface.
<code>tools/train.py</code>	Builds datasets, trains, validates, checkpoints, and writes metrics.
<code>data/</code>	Dataset loaders for PACS, VLCS, and Office-Home.
<code>configs/</code>	Default hyperparameters for each dataset and method.
<code>scripts/</code>	Shell scripts for full experiment grids.
<code>reportcv/</code>	This report and generated result tables.

5.2 Model Construction

The local implementation defines three model families. `ERMResNet` keeps the standard ResNet classifier and returns logits with an empty feature list. `OfficialM2` removes the final ResNet fully-connected layer, registers forward hooks at selected intermediate layers, passes the captured activations through extraction blocks, concatenates the projected outputs, and classifies the resulting hypercolumn-like vector. `M2CL` is a backward-compatible name for the same architecture with intermediate energy activations returned for the contrastive loss.

For ResNet-18, extraction blocks are attached to all 13 convolutional and residual outputs across `layer1` through `layer4`. For ResNet-50, extraction blocks are connected to the last convolutional layer of each bottleneck block.

Forward hooks are used to collect intermediate tensors without manually rewriting the ResNet forward pass. The tradeoff is that the feature dictionary must be cleared before each forward pass, and hooks must be removed if the model is discarded.

5.3 Extraction Block Details

The extraction block supports two modes: parallel and cascading. In the *parallel* mode, each concentration pipeline has its own 1×1 convolution before pooling. In the *cascading* mode, all pipelines share a single compressed feature map. The project uses the parallel configuration because the ablation study (Section 9) shows stronger overall performance.

5.4 Loss Implementation

The loss module in `losses/contrastive.py` computes cross-entropy on the final logits. For M^2 , the contrastive weight α is effectively zero. For M^2 -CL, the module computes S_l by summing $\log(p_c^{(l)})$ over classes with at least one positive pair in the batch, as defined in Eq. (4). The combined loss is therefore:

$$\mathcal{L}_{\text{impl}} = \mathcal{L}_{\text{CE}} - \alpha \sum_l S_l = \mathcal{L}_{\text{CE}} + \alpha \sum_l \mathcal{L}^{(l)}, \quad (7)$$

which is equivalent to Eq. (6), since $S_l = -\mathcal{L}^{(l)}$.

This implementation has a direct practical implication: the quality of the contrastive signal depends on the batch containing enough same-class pairs. With small batches or datasets with many classes, some classes may appear only once in a batch, contributing no positive-pair signal. This is one reason the default batch size is 128.

5.5 Training Pipeline

The training entry point loads dataset defaults from configuration files, then resolves command-line overrides. `build_datasets` constructs a leave-one-domain-out split: the chosen test domain becomes the unseen target, and the remaining domains become source environments.

A DomainBed-style source validation split is applied: each source environment is split with a 0.2 holdout fraction using a seeded hash. Model selection uses source validation accuracy rather than target-domain test accuracy, which would violate the DG protocol.

During training, `M2Algorithm.update` concatenates mini-batches across source environments, runs the model, computes the combined objective, and updates the optimizer. After training, the best checkpoint is reloaded and evaluated on the held-out target domain; metrics are saved as JSON files for later aggregation.

5.6 Configuration Defaults

Project defaults for M^2 and M^2 -CL: ImageNet-pretrained ResNet-18 or ResNet-50, SGD optimizer, learning rate 0.001, 30 epochs, batch size 128, reduction ratio $r = 4$, dropout probability 0.3, $\alpha = 0.01$, $\tau = 1.0$. Baseline methods use separate DomainBed-style defaults because they have different optimization behavior.

6. Project Workflow

The repository expects datasets to be placed under a common data root. `tools/check_data.py` validates folder layout before long runs. A single PACS run with M^2 -CL is launched as:

```
python tools/train.py -dataset pacs -test_domain photo -method m2cl
```

VLCS and Office-Home follow the same pattern with different dataset and domain arguments. For full experiment grids, `tools/run_experiments.py` iterates over datasets, target domains, methods, backbones, and seeds.

7. Experimental Setup

7.1 Datasets

The project uses three DG benchmarks:

- **PACS** [13]: 9,991 images, 7 classes, 4 domains (Photo, Art Painting, Cartoon, Sketch).
- **VLCS** [14]: 10,729 images, 5 classes, 4 domains (PASCAL VOC, LabelMe, Caltech101, SUN09).
- **Office-Home** [15]: 15,588 images, 65 classes, 4 domains (Art, Clipart, Product, Real-World).



Figure 4: Datasets used in the project: PACS, VLCS, and Office-Home. Each dataset presents a distinct type of domain shift.

7.2 Training Configuration

- Backbones: ResNet-18 and ResNet-50 pretrained on ImageNet.
- Optimizer: SGD with learning rate 0.001.
- Training length: 30 epochs per domain split.
- Batch size: 128 where GPU memory allows.
- Extraction reduction factor: $r = 4$.
- Pipelines per extraction block: 3 (early layers) and 2 (later layers).
- Pool sizes: $k \in \{8, 4, 2\}$ (early), $k \in \{7, 3\}$ (later).
- Contrastive loss weight: $\alpha = 0.01$.
- Temperature: $\tau = 1.0$.

7.3 Baselines

ERM, RSC [3], CORAL [4], Mixup [6], MMD [5], SagNet [7], SelfReg [8], ARM [9], EQRM [10], and SAGM [11]. These cover standard supervised learning, representation suppression, feature distribution alignment, sample interpolation, style robustness, self-supervised regularization, adaptive risk learning, and sharpness-aware generalization. All baselines are run using a common DomainBed-style experimental interface.

8. Quantitative Results

8.1 Summary Results

Table 3 presents the main benchmark averages. Both backbones show a consistent two-step gain: M² already outperforms the strongest competing method, and adding the contrastive regularizer

in M²-CL pushes the numbers further. The gain is largest on Office-Home with ResNet-50, where the stronger backbone benefits from the richer set of intermediate representations.

Table 3: Average top-1 accuracy and gain over the best baseline. Higher is better. Δ is gain over the best competing method.

Backbone	Method	PACS	Δ	VLCS	Δ	Off.-Home	Δ
ResNet-18	Best baseline	81.07	–	77.57	–	63.35	–
ResNet-18	M ²	81.81	+0.74	78.00	+0.43	63.47	+0.12
ResNet-18	M²-CL	82.51	+1.44	78.50	+0.93	64.18	+0.83
ResNet-50	Best baseline	84.89	–	78.10	–	67.33	–
ResNet-50	M ²	86.11	+1.22	78.76	+0.66	68.70	+1.37
ResNet-50	M²-CL	86.56	+1.67	79.09	+0.99	69.34	+2.01

8.2 Breakdown by Comparison

Table 4: Compact interpretation of the project experimental results.

Comparison	R18 Avg Gain	R50 Avg Gain	Interpretation
M ² vs. best baseline	+0.43	+1.08	Multilayer/multiscale features help even without contrastive loss.
M ² -CL vs. best baseline	+1.07	+1.56	Full method gives consistent average improvement.
M ² -CL vs. M ²	+0.64	+0.47	Contrastive loss adds a stable extra gain on top of the architecture.

8.3 Per-Dataset Interpretation

Table 5: Dataset-level reading of the results.

Dataset	Main shift type	Observed behavior	Practical comment
PACS	Style and texture shift	M ² -CL improves on both backbones.	Good benchmark for sketch/cartoon robustness.
VLCS	Dataset-source bias	Gain is positive but smaller than PACS.	Consistent with DG literature on subtle cross-dataset shifts.
Office-Home	Many classes, style variation	Strongest ResNet-50 gain.	Richer backbone benefits most from intermediate representations.

Each held-out domain introduces a different kind of shift, so per-domain numbers naturally vary. Sketch-like data is especially difficult because texture and color information are largely absent, leaving only shape cues. Office-Home is challenging due to the large number of classes (65), but the ResNet-50 backbone handles this well.

9. Ablation and Sensitivity Analysis

The ablation study isolates four design choices: extraction pipeline type (cascading vs. parallel), reduction factor r , spatial dropout, and the contrastive loss. All runs use ResNet-18 on PACS and VLCS.

The ablation and sensitivity tables should be read as independent controlled sweeps. Their purpose is to compare variants within the same sweep, not to serve as exact reruns of the main benchmark table. Therefore, rows that share the same nominal default values may differ from Table 3 because they come from separate experiment grids.

The best configuration in the project is: parallel pipeline, $r = 4$, spatial dropout enabled, and contrastive loss active. Adding the contrastive loss to this configuration improves PACS from 81.81 to 82.51 and VLCS from 78.00 to 78.50 in the ablation setting.

Table 6: Interpretation of the ablation and sensitivity study.

Design choice	Tested variants	Best setting	Interpretation
Pipeline layout	cascading, parallel	parallel	Separate scale-specific pipelines preserve richer evidence.
Reduction ratio r	2, 4, 6	4	Balances compactness and information retention.
Spatial dropout	with / without	with dropout	Drops correlated feature maps, reducing shortcut dependence.
Loss weight α	0 to 10^{-1}	small nonzero value	Helpful as regularizer; harmful when it dominates CE.
Temperature τ	0.01 to 100	near 1.0 (default)	Controls sharpness of pair-similarity distribution.

9.1 Key Lessons from the Ablation

The parallel pipeline consistently outperforms the cascading variant. Sharing a single compressed feature map across all scales saves parameters, but each scale ends up constrained to the same intermediate representation — which limits what finer or coarser pooling can actually capture. Giving each pipeline its own 1×1 convolution removes that bottleneck. The reduction ratio $r = 4$ sits at a reasonable trade-off: going lower lets more domain-specific texture through, going higher starts discarding class-relevant detail.

Spatial dropout turns out to matter more here than in standard classification. Because adjacent activations in a convolutional map are correlated, scalar dropout can remove isolated values while leaving nearby evidence intact — enough for a shortcut to survive. Dropping entire channels is more disruptive in the right way. On the contrastive weight: α must stay small. At $\alpha = 10^{-1}$ the contrastive term dominates cross-entropy and accuracy on PACS drops by over 15 points. The loss is meant to nudge the representation, not take over from the classifier.

10. Qualitative Results

The qualitative analysis uses gradient saliency maps to visualize which pixels contribute most strongly to the model’s prediction. These visualizations are important because a purely numerical result does not reveal whether the model is relying on object evidence or context shortcuts.

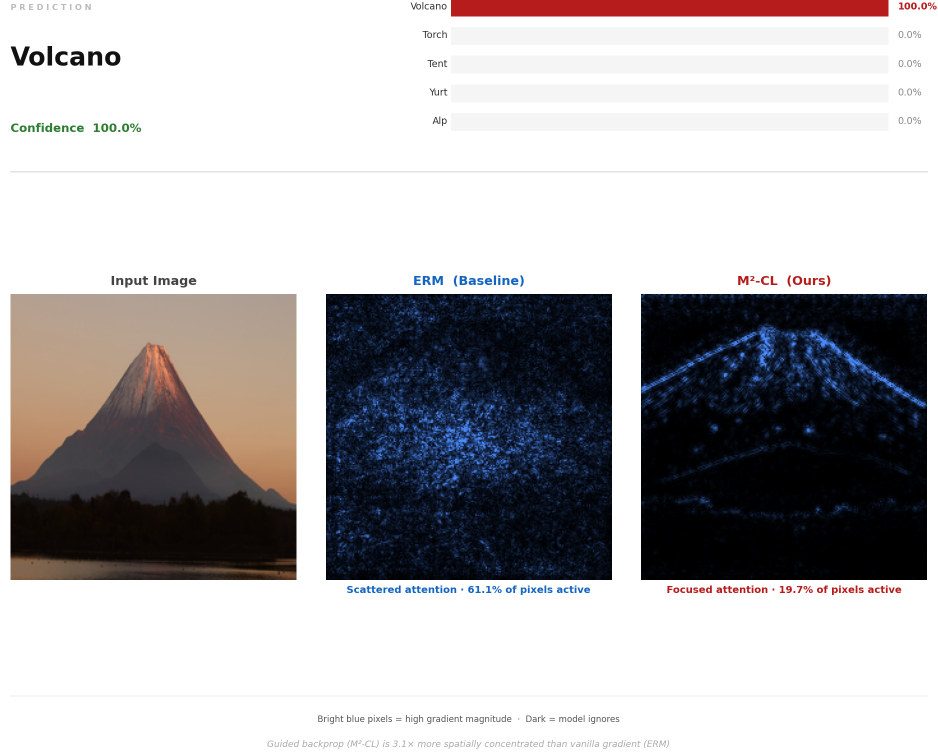


Figure 5: PACS saliency maps comparing a baseline (ERM ResNet-18) and M²-CL. Darker regions indicate pixels with stronger influence on the predicted class. M²-CL focuses more on object regions and boundaries; ERM can assign significant weight to domain-specific background.

ERM tends to assign high gradient weight to background regions and domain-specific context, while M²-CL concentrates on object boundaries and discriminative parts. This matches what the numbers suggest: suppressing domain-specific shortcuts at the representation level leads to better generalization when the test domain changes.

11. Discussion

The quantitative results across three benchmarks support the core design idea: pulling representations from intermediate layers gives the classifier access to both local and global visual evidence that would otherwise be collapsed into the final feature vector. The contrastive regularizer adds consistent improvement on top, though the sensitivity analysis shows it needs to stay at a small weight.

On the implementation side, the forward-hook approach keeps the code modular — the backbone is unchanged, and extraction blocks plug in at selected layers. Wiring up a common interface for all baselines was important for fair comparison, since DG methods tend to have different default hyperparameters and training dynamics. Memory usage grows noticeably with ResNet-50 and batch size 128, which is worth accounting for when running the full experiment grid.

12. Limitations

Running M²-CL at full scale requires careful memory management: concatenating intermediate vectors from 13 extraction blocks (ResNet-18) or more (ResNet-50) adds up quickly, and the pairwise contrastive computation scales with batch size. In practice we kept batch size at 128 to balance GPU memory and contrastive signal quality.

13. Conclusion

We implemented and evaluated M²-CL for domain generalization across three benchmarks. The method extracts intermediate CNN representations at multiple scales and applies a supervised contrastive regularizer (Eq. 3–6) to promote class-level invariance across source domains.

On PACS, VLCS, and Office-Home, M²-CL improves over both the best competing baseline and the M² backbone under ResNet-18 and ResNet-50. The gains are consistent across datasets, and the saliency maps show the model attending to object parts rather than domain-specific background — which is exactly the behavior the contrastive objective is designed to encourage.

A. Full Result Tables

The following pages provide the full result tables from the project experiments. Tables for the main benchmark results and tables for ablation/sensitivity sweeps play different roles: the main tables report the final benchmark comparison, while the sweep tables analyze trends under separately executed controlled runs.

Result Tables

Table 1: Top-1 accuracy on PACS and VLCS, ResNet-18.

Method	Art	Cartoon	Photo	Sketch	Avg	Caltech	Labelme	Sun	Voc	Avg
ERM	76.53	74.80	94.95	67.89	78.54	96.87	65.03	71.62	71.60	76.28
RSC	79.26	76.12	95.07	70.88	80.34	97.68	66.33	73.53	72.75	77.57
CORAL	78.55	72.74	92.24	73.93	79.36	95.16	60.73	70.66	65.00	72.89
MIXUP	79.79	72.81	93.11	63.97	77.42	93.44	60.25	74.36	69.87	74.48
MMD	77.75	73.35	92.34	69.74	78.29	97.28	60.62	70.48	65.21	73.40
SagNet	78.71	74.90	94.79	70.48	79.72	95.76	61.22	69.61	71.90	74.62
SelfReg	79.46	76.32	95.93	71.60	80.83	92.53	62.56	70.27	72.15	74.38
ARM	79.77	76.33	94.43	74.76	81.07	95.86	60.19	70.66	72.83	74.89
EQRM	78.61	72.38	92.48	71.13	78.65	96.17	60.57	70.44	64.66	72.96
SAGM	78.22	75.14	94.79	70.46	79.65	96.37	66.22	71.14	71.35	76.27
M2	<u>80.64</u>	<u>76.91</u>	95.50	74.20	<u>81.81</u>	<u>98.24</u>	<u>67.02</u>	73.80	<u>72.92</u>	<u>78.00</u>
M2-CL	81.04	77.48	<u>95.70</u>	75.81	82.51	98.69	67.48	<u>74.05</u>	73.76	78.50

Table 2: Top-1 accuracy on PACS and VLCS, ResNet-50.

Method	Art	Cartoon	Photo	Sketch	Avg	Caltech	Labelme	Sun	Voc	Avg
ERM	83.54	78.47	96.51	73.64	83.04	97.01	62.22	71.79	76.09	76.78
RSC	82.08	80.26	96.51	75.45	83.57	97.81	64.44	74.60	75.53	78.10
CORAL	83.94	79.27	94.37	81.98	84.89	97.01	61.34	75.26	71.74	76.34
MIXUP	85.16	77.40	96.23	78.26	84.26	96.94	62.33	73.40	72.32	76.25
MMD	82.21	74.70	92.89	69.00	79.70	97.48	63.85	75.49	71.65	77.12
SagNet	85.23	78.47	93.20	77.71	83.65	94.14	60.43	70.49	69.96	73.75
SelfReg	83.33	79.06	93.16	77.24	83.20	92.49	60.73	66.80	73.58	73.40
ARM	82.14	79.10	92.97	73.02	81.81	96.01	63.98	66.26	71.79	74.51
EQRM	84.98	77.74	91.26	74.88	82.21	94.35	60.80	66.35	72.94	73.61
SAGM	81.35	78.99	93.24	79.39	83.24	97.39	64.23	69.30	76.17	76.77
M2	<u>86.02</u>	79.90	<u>97.03</u>	81.50	<u>86.11</u>	97.52	<u>64.77</u>	<u>76.02</u>	<u>76.72</u>	<u>78.76</u>
M2-CL	86.48	<u>80.05</u>	97.99	<u>81.70</u>	86.56	<u>97.60</u>	64.86	76.65	77.26	79.09

Table 3: Top-1 accuracy on Office-Home, ResNet-18.

Method	Art	Clipart	Product	Real-World	Avg
ERM	56.12	47.88	<u>72.47</u>	73.58	62.51
RSC	56.00	46.09	70.85	72.32	61.31
CORAL	55.46	49.39	71.75	73.28	62.47
MIXUP	56.37	<u>49.81</u>	70.87	73.61	62.66
MMD	55.67	<u>48.52</u>	71.68	73.19	62.27
SagNet	55.75	47.74	71.71	73.17	62.09
SelfReg	57.56	49.69	72.43	73.72	63.35
ARM	54.47	48.45	71.14	<u>73.79</u>	61.96
EQRM	56.08	48.34	71.57	<u>72.94</u>	62.23
SAGM	55.79	46.51	71.16	73.15	61.65
M2	<u>58.04</u>	49.30	73.04	73.50	<u>63.47</u>
M2-CL	58.77	50.86	72.20	74.91	64.18

Figure 6: Full result tables, page 1: PACS (ResNet-18 and ResNet-50), VLCS (ResNet-18 and ResNet-50).

Table 4: Top-1 accuracy on Office-Home, ResNet-50.

Method	Art	Clipart	Product	Real-World	Avg
ERM	63.43	51.32	76.45	78.13	67.33
RSC	62.64	51.55	75.11	77.34	66.66
CORAL	61.27	54.34	73.35	77.83	66.70
MIXUP	63.23	51.91	77.10	76.39	67.16
MMD	59.63	49.37	75.28	74.71	64.75
SagNet	61.41	51.85	72.85	73.79	64.97
SelfReg	59.95	51.22	71.28	74.52	64.24
ARM	54.23	52.14	71.73	71.26	62.34
EQRM	63.52	52.32	73.28	76.45	66.39
SAGM	61.01	53.27	74.81	77.00	66.52
M2	65.35	53.90	76.80	78.74	68.70
M2-CL	66.46	54.82	78.24	77.85	69.34

Table 5: Independent ablation sweep on PACS and VLCS, ResNet-18.

pipe	r	drop	loss	A	C	P	S	PACS Avg	C	L	S	V	VLCS Avg
c	2	-	-	72.34	72.29	94.19	69.51	77.08	93.87	60.91	65.89	61.91	70.65
c	4	-	-	74.38	72.74	90.13	69.71	76.74	94.14	59.01	67.82	65.42	71.60
c	6	-	-	75.72	70.12	94.26	73.33	78.36	91.93	56.81	67.28	64.10	70.03
c	2	yes	-	75.67	73.29	92.16	75.95	79.27	93.73	57.72	66.34	62.50	70.07
c	4	yes	-	74.36	74.05	92.57	76.45	79.36	96.07	56.28	67.07	64.30	70.93
c	6	yes	-	75.97	72.77	93.01	74.73	79.12	92.94	59.73	66.15	64.51	70.83
p	2	-	-	76.09	55.88	68.35	60.92	65.31	95.60	56.97	66.25	73.15	72.99
p	4	-	-	74.72	74.62	92.01	70.32	77.92	95.20	58.10	66.20	64.90	71.10
p	6	-	-	75.38	69.99	91.21	69.47	76.51	95.23	56.81	69.13	65.34	71.63
p	2	yes	-	74.61	72.39	87.08	73.95	77.01	95.73	58.33	68.44	63.74	71.56
p	6	yes	-	76.73	73.98	93.41	73.56	79.42	93.19	56.05	66.62	64.24	70.03
p	4	yes	-	80.64	76.91	95.50	74.20	81.81	98.24	67.02	73.80	72.92	78.00
p	4	yes	yes	81.04	77.48	95.70	75.81	82.51	98.69	67.48	74.05	73.76	78.50

Table 6: Independent sensitivity sweep for τ in M2-CL, ResNet-18.

τ	PACS	VLCS
0.01	78.10	75.95
0.1	80.03	77.27
0.2	81.37	76.71
0.4	82.07	77.03
0.6	82.30	77.65
0.8	82.44	78.12
1.0	82.51	78.50
1.2	82.18	78.05
1.4	81.74	77.69
1.6	81.21	77.12
1.8	80.51	76.66
2.0	78.72	75.99
10.0	79.92	74.04
100.0	78.84	73.74

Table 7: Independent sensitivity sweep for α in M2-CL, ResNet-18.

α	PACS	VLCS
0.00	81.81	78.00
10^{-5}	81.95	78.10
10^{-4}	82.18	78.22
10^{-3}	82.29	78.34
10^{-2}	82.51	78.50
10^{-1}	64.04	55.96

Figure 7: Full result tables, page 2: Office-Home (ResNet-18 and ResNet-50), ablation study on PACS and VLCS, sensitivity analysis for τ and α .

B. References

References

- [1] A. Ballas and C. Diou, “Multiscale and Multilayer Contrastive Learning for Domain Generalization,” *IEEE Transactions on Artificial Intelligence*, vol. 5, 2024. DOI: 10.1109/TAI.2024.3377173.
- [2] I. Gulrajani and D. Lopez-Paz, “In Search of Lost Domain Generalization,” *Int. Conf. on Learning Representations (ICLR)*, 2021.
- [3] Z. Huang, H. Wang, E. P. Xing, and D. Huang, “Self-Challenging Improves Cross-Domain Generalization,” *European Conf. on Computer Vision (ECCV)*, 2020.
- [4] B. Sun and K. Saenko, “Deep CORAL: Correlation Alignment for Deep Domain Adaptation,” *European Conf. on Computer Vision Workshops (ECCVW)*, 2016.
- [5] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Scholkopf, and A. Smola, “A Kernel Two-Sample Test,” *Journal of Machine Learning Research*, vol. 13, pp. 723–773, 2012.
- [6] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, “mixup: Beyond Empirical Risk Minimization,” *Int. Conf. on Learning Representations (ICLR)*, 2018.
- [7] H. Nam, H. Lee, J. Park, W. Yoon, and D. Yoo, “Reducing Domain Gap by Reducing Style Bias,” *arXiv:1910.11645*, 2019.
- [8] D. Kim, S. Park, J. Kim, and J. Lee, “SelfReg: Self-supervised Contrastive Regularization for Domain Generalization,” *arXiv:2104.09841*, 2021.
- [9] M. Zhang, H. Marklund, N. Dhawan, A. Gupta, S. Levine, and C. Finn, “Adaptive Risk Minimization: Learning to Adapt to Domain Shift,” *arXiv:2007.02931*, 2020.
- [10] C. Eastwood, A. Robey, S. Singh, J. von Kugelgen, H. Hassani, G. J. Pappas, and B. Scholkopf, “Probable Domain Generalization via Quantile Risk Minimization,” *arXiv:2207.09944*, 2022.
- [11] P. Wang, Z. Zhang, Z. Lei, and L. Zhang, “Sharpness-Aware Gradient Matching for Domain Generalization,” *arXiv:2303.10353*, 2023.
- [12] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” *IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [13] D. Li, Y. Yang, Y.-Z. Song, and T. M. Hospedales, “Deeper, Broader and Artier Domain Generalization,” *IEEE Int. Conf. on Computer Vision (ICCV)*, 2017.
- [14] A. Torralba and A. A. Efros, “Unbiased Look at Dataset Bias,” *IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2011.
- [15] H. Venkateswara, J. Eusebio, S. Chakraborty, and S. Panchanathan, “Deep Hashing Network for Unsupervised Domain Adaptation,” *IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2017.